

IntelligenceOS Product Architecture and Platform Specification

IntelligenceOS Product Architecture & Platform Specification

1. Executive Overview

IntelligenceOS is an enterprise-grade Autonomous Copilot and Retrieval-Augmented Generation (RAG) platform. It provides organizations with unified knowledge indexing, dynamic prompt orchestration, autonomous ReAct agent execution, continuous LLM evaluation, and full-lifecycle observability.

2. Core Architecture Topology

The platform is structured into decoupled, independently scalable components:

- **FastAPI Web Service**: High-performance asynchronous REST API handling authentication, workspace management, document and URL source registrations, chat session routing, and evaluation triggers.
- **Background Worker Engine** (`app.worker`): Autonomous Python worker process consuming ingestion and indexing tasks from a Redis queue broker. Performs document parsing, semantic chunking, embedding generation, and vector upsertion.
- **Relational Metadata Store** (PostgreSQL 16): Houses tenant workspaces, users, source statuses, conversation histories, agent execution logs, and evaluation metrics with complete referential integrity.
- **Vector Database** (Qdrant Cloud): Distributed vector database storing semantic chunk embeddings with HNSW indexing and payload-level multi-tenant isolation.
- **Object Storage** (Supabase Storage): Secure cloud bucket (`IntelligenceOS(Files)`) storing raw uploaded documents (PDF, TXT, Markdown, CSV, DOCX).
- **In-Memory Cache & Message Broker** (Redis): High-throughput broker managing background queues (`default`) and distributed rate limiting.
- **Frontend Single-Page Application** (React 18 + Vite): Modern dashboard featuring responsive Dark/Light themes, Lucide icon system, and client-side routing.

3. Multi-Tenant Isolation

Tenant isolation is enforced across every layer:

- In PostgreSQL, foreign keys link all resources to a unique `workspace_id`.
- In Qdrant, all vector queries apply strict payload filters:

```
```json
{"key": "workspace_id", "match": {"value": "<workspace_id>"}}
```
```

Vectors from one workspace can never be accessed or retrieved by queries executed in a different workspace.

4. Ingestion & Indexing Pipeline

Document ingestion follows an asynchronous pipeline:

1. **Source Registration**: A user submits a file upload or web URL via the UI.
2. **Storage Persistence**: Uploaded files are streamed to Supabase Storage; URL requests are registered in PostgreSQL.
3. **Queue Enqueue**: An ingestion job `source_id` is enqueued into the Redis task queue.

4. **Worker Extraction**: The worker downloads the file or crawls the URL HTML, normalizing text content.
5. **Semantic Chunking**: Content is split into chunks of 500 tokens with a 50-token overlap to preserve semantic context across chunk boundaries.
6. **Vector Embedding**: Each chunk is transformed into a dense vector embedding using Google Gemini / OpenAI embedding models.
7. **Qdrant Indexing**: Vectors with chunk metadata (`title`, `source\_id`, `chunk\_index`, `workspace\_id`) are batch-upserted to Qdrant.
8. **Status Transition**: Source status moves from `pending` -> `processing` -> `completed` (or `failed` upon unrecoverable error).

5. RAG Retrieval & Prompt Routing

When a conversation message is sent:

- Query expansion and semantic vector similarity search retrieve the top-K relevant chunks (default: K=5, threshold=0.65).
- A dynamic grounded prompt compiles the retrieved snippets with strict citation references formatted as `[source\_id:chunk\_idx]`.
- If no matching chunks exceed the relevance threshold, the system provides a graceful fallback rather than hallucinating answers.